

Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Факультет Программной Инженерии и Компьютерной Техники

Лабораторная работа №2

По дисциплине «**Бизнес-логика программных систем**»

Вариант №91

Выполнили:

Студенты группы Р3306,

Михайлов Дмитрий Андреевич

Медведев Владислав Александрович

Преподаватель:

Кривоносов Егор Дмитриевич



Санкт-Петербург

2026 год

Содержание

Задание	3
Модель потока управления для автоматизируемого бизнес-процесса	4
Спецификация пользовательских привилегий и ролей, реализованных в приложении.....	4
UML-диаграммы классов и пакетов разработанного приложения	6
Спецификация REST API для всех публичных интерфейсов разработанного приложения	10
1. Courses API	10
3. Payments API.....	14
4. Payment Webhook API.....	15
5. Сводная таблица публичных endpoint-ов	16
Исходный код системы или ссылка на репозиторий с исходным кодом	16
Вывод по работе.....	17

Задание

Доработать приложение из лабораторной работы #1, реализовав в нём управление транзакциями и разграничение доступа к операциям бизнес-логики в соответствии с заданной политикой доступа.

Управление транзакциями необходимо реализовать следующим образом:

1. Переработать согласованные с преподавателем прецеденты (или по согласованию с ним разработать новые), объединив взаимозависимые операции в рамках транзакций.
2. Управление транзакциями необходимо реализовать с помощью Spring JTA.
3. В реализованных (или модифицированных) прецедентах необходимо использовать программное управление транзакциями.
4. В качестве менеджера транзакций необходимо использовать Atomikos.

Разграничение доступа к операциям необходимо реализовать следующим образом:

1. Разработать, специфицировать и согласовать с преподавателем набор привилегий, в соответствии с которыми будет разграничиваться доступ к операциям.
2. Специфицировать и согласовать с преподавателем набор ролей, осуществляющих доступ к операциям бизнес-логики приложения.
3. Реализовать разработанную модель разграничений доступа к операциям бизнес-логики на базе Spring Security + JAAS. Информацию об учётных записях пользователей необходимо сохранять в реляционную базу данных, для аутентификации использовать HTTP basic.

Правила выполнения работы:

1. Все изменения, внесённые в реализуемый бизнес-процесс, должны быть учтены в описывающей его модели, REST API и наборе скриптов для тестирования публичных интерфейсов модуля.
2. Доработанное приложение необходимо развернуть на сервере helios.

Содержание отчёта:

1. Текст задания.
2. Модель потока управления для автоматизируемого бизнес-процесса.
3. Спецификация пользовательских привилегий и ролей, реализованных в приложении.
4. UML-диаграммы классов и пакетов разработанного приложения.
5. Спецификация REST API для всех публичных интерфейсов разработанного приложения.
6. Исходный код системы или ссылка на репозиторий с исходным кодом.
7. Выводы по работе.

Модель потока управления для автоматизируемого бизнес-процесса

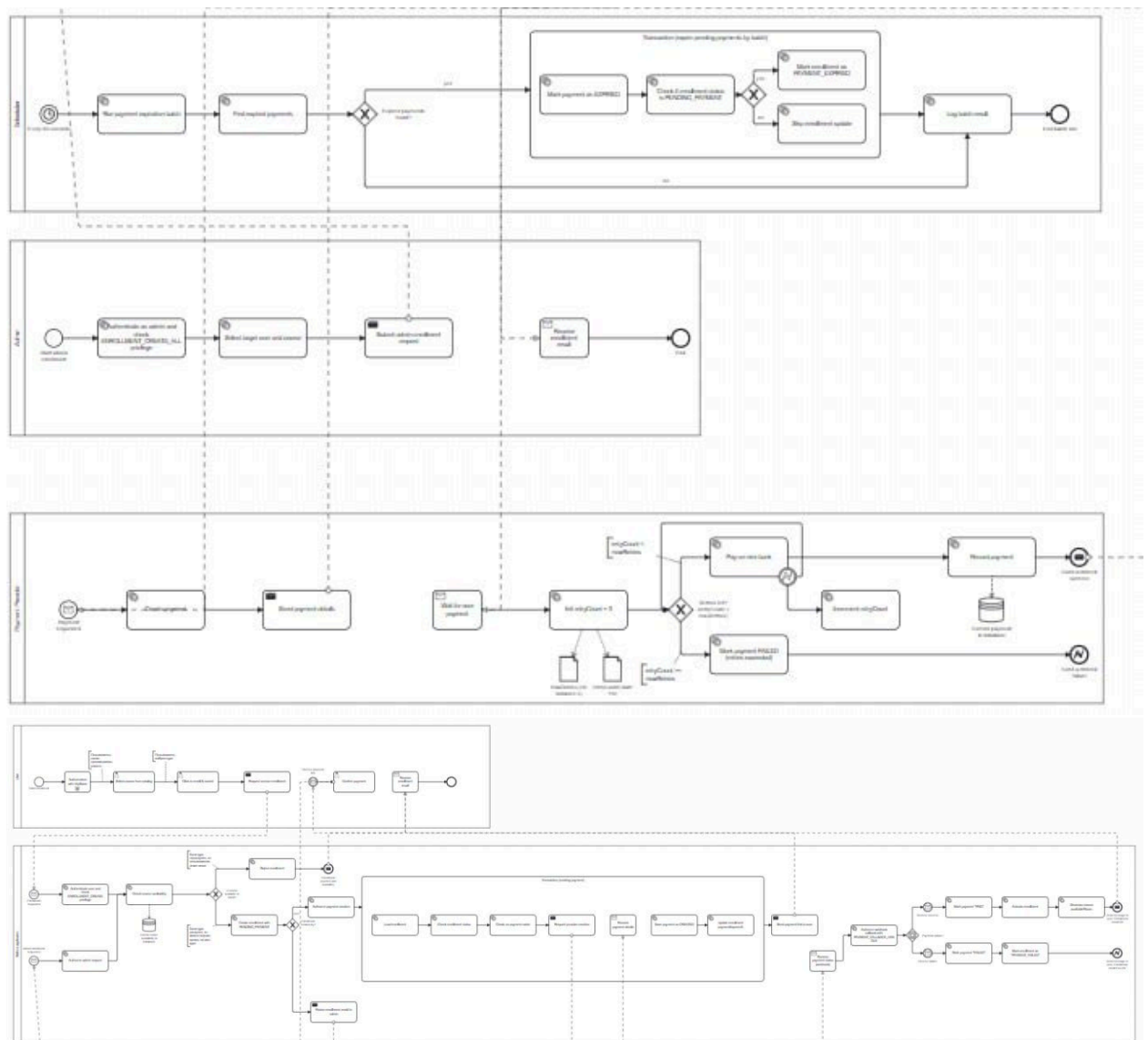


Рис. 1. BPMN диаграмма бизнес-процесса - запись на курс (с оплатой).

Спецификация пользовательских привилегий и ролей, реализованных в приложении.

Привилегия	Назначение	Операции
COURSE_READ	Просмотр информации о курсах	Получение списка курсов, получение курса по идентификатору, фильтрация курсов
ENROLLMENT_CREATE	Создание собственной записи на курс	Создание заявки на запись текущего аутентифицированного

Привилегия	Назначение	Операции
		пользователя на выбранный курс
ENROLLMENT_CREATE_ALL	Административное создание записи на курс	Создание записи на курс для произвольного пользователя
ENROLLMENT_READ_ALL	Просмотр всех записей на курсы	Получение записи по идентификатору без ограничения владельцем, просмотр записей пользователя, просмотр записей по курсу
PAYMENT_CREATE	Создание платежа по собственной записи	Создание платежа для enrollment, принадлежащего текущему пользователю
PAYMENT_READ_ALL	Просмотр всех платежей	Получение платежа по идентификатору или по enrollment без ограничения владельцем
PAYMENT_CALLBACK_HANDLE	Обработка callback-запросов от платёжного сервиса	Приём webhook-запросов со статусами PAID и FAILED

Табл. 1. Пользовательские привилегии.

Роль	Назначение	Набор привилегий
USER	Обычный пользователь системы, который может просматривать доступные курсы, записываться на курс и оплачивать собственную запись	COURSE_READ, ENROLLMENT_CREATE, PAYMENT_CREATE
ADMIN	Администратор системы, имеющий доступ к административным операциям и просмотру данных всех пользователей	COURSE_READ, ENROLLMENT_CREATE, ENROLLMENT_CREATE_ALL, ENROLLMENT_READ_ALL, PAYMENT_CREATE, PAYMENT_READ_ALL, PAYMENT_CALLBACK_HANDLE

Табл. 2. Пользовательские роли.

Доступ к операциям бизнес-логики ограничивается не только набором привилегий, но и проверкой владения ресурсом. Например, пользователь с привилегией PAYMENT_CREATE может создать платёж только для своей записи на курс. Для этого используется сервис AccessService, который сравнивает email текущего пользователя с email владельца соответствующего enrollment или payment. Администратор с ролью ADMIN проходит такие проверки без ограничения владельцем ресурса.

Для операций с платежами применяются следующие правила, что создание платежа требует привилегии PAYMENT_CREATE и проверки принадлежности enrollment текущему пользователю; чтение платежа доступно либо пользователю с PAYMENT_READ_ALL, либо владельцу соответствующего платежа.

Для операций с записями на курсы применяются следующие правила, что обычный пользователь может создать запись только для себя, администратор может создать запись для любого пользователя, а просмотр всех записей по курсу доступен только пользователям с привилегией ENROLLMENT_READ_ALL.

UML-диаграммы классов и пакетов разработанного приложения

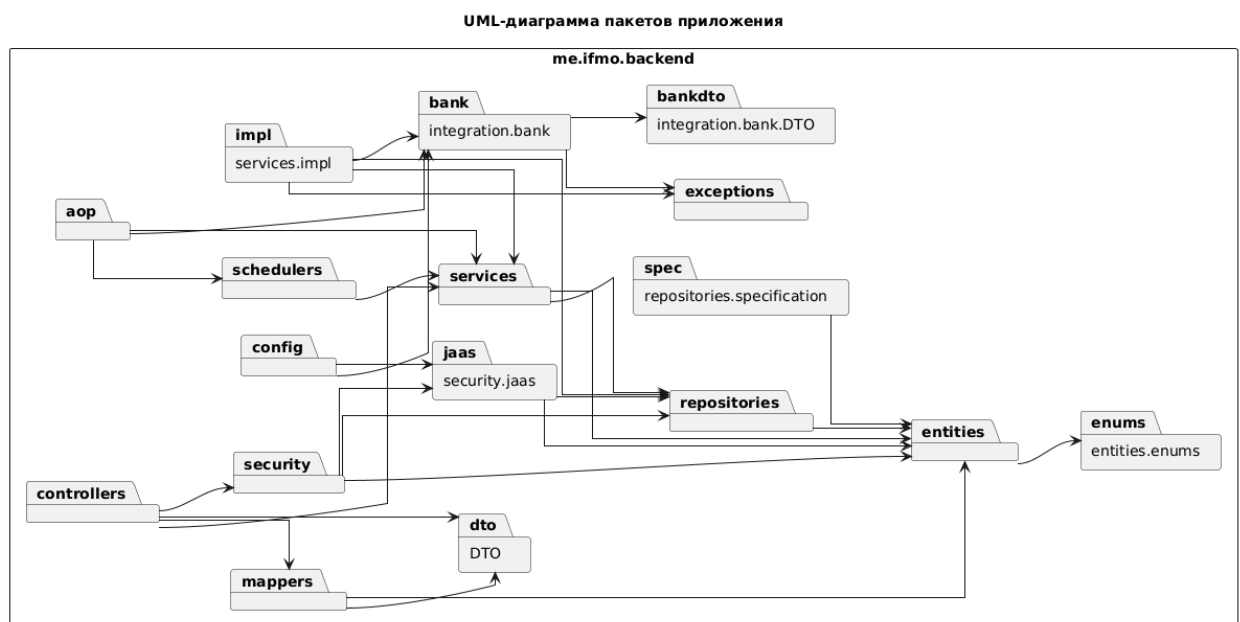


Рис. 2. UML диаграмма пакетов.

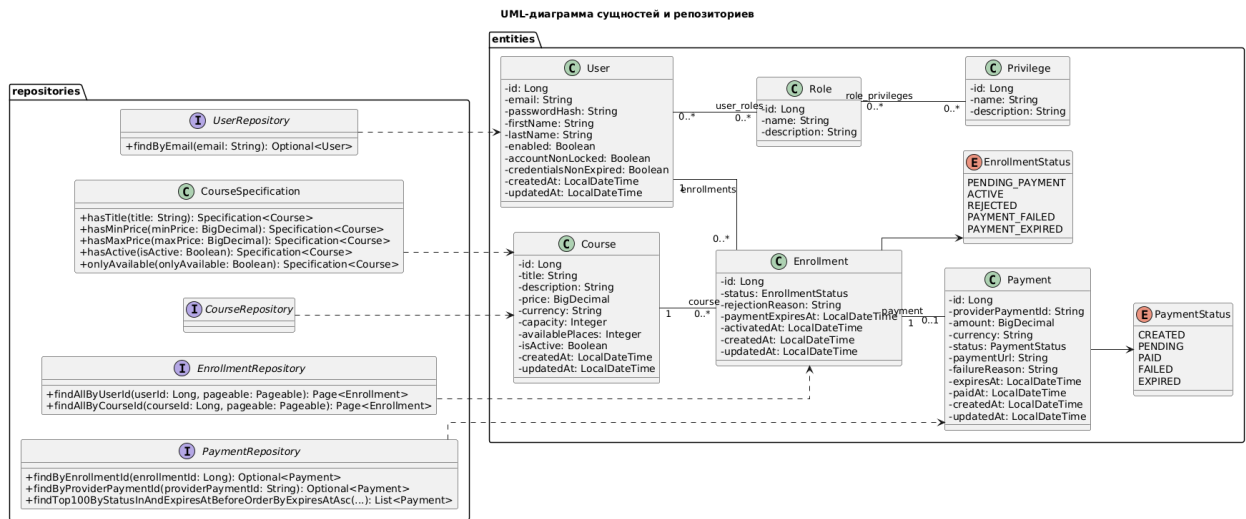


Рис. 3. UML диаграмма классов сущностей и репозитория.

UML-диаграмма классов бизнес-логики

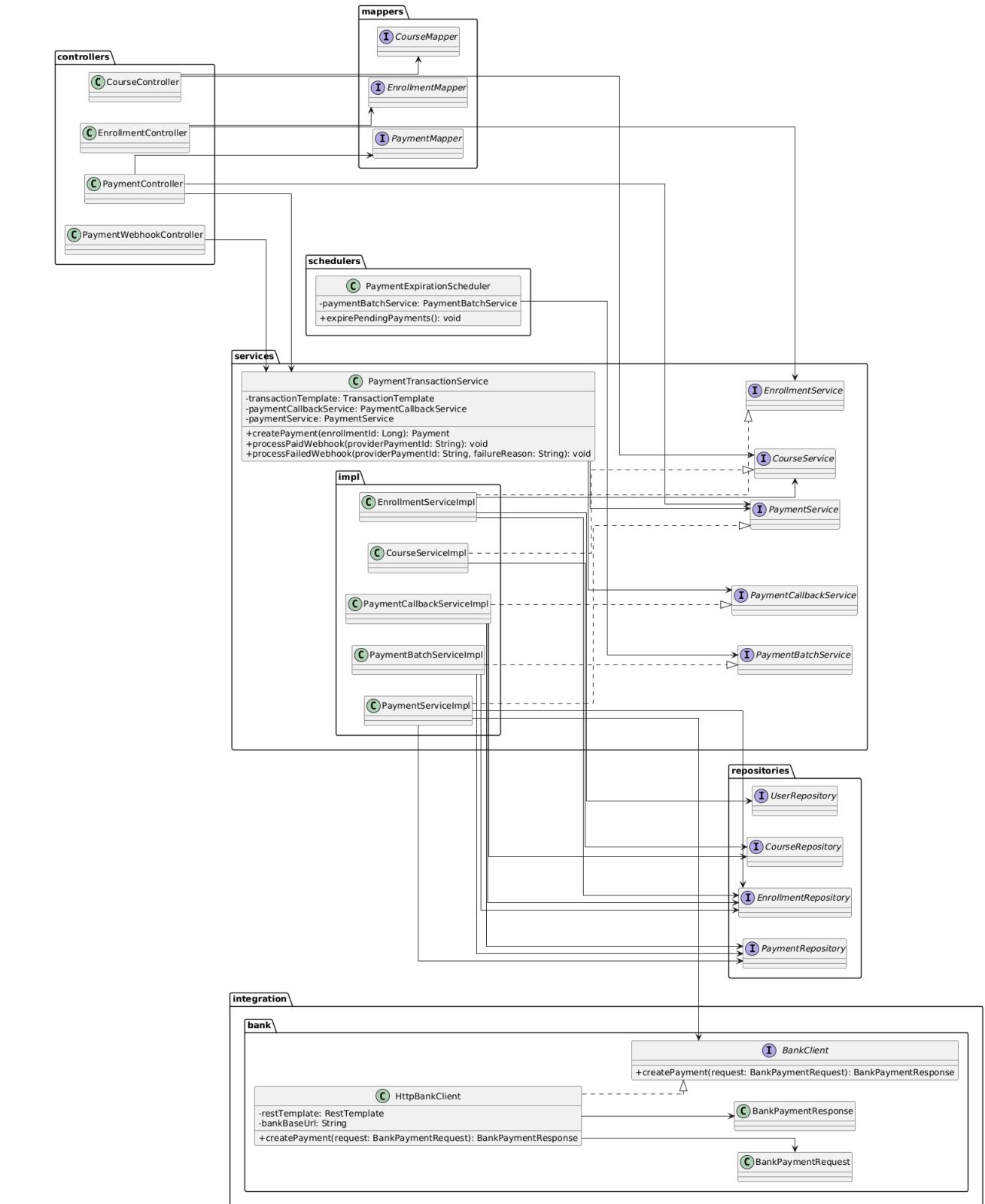


Рис. 4. UML диаграмма классов бизнес-логики.

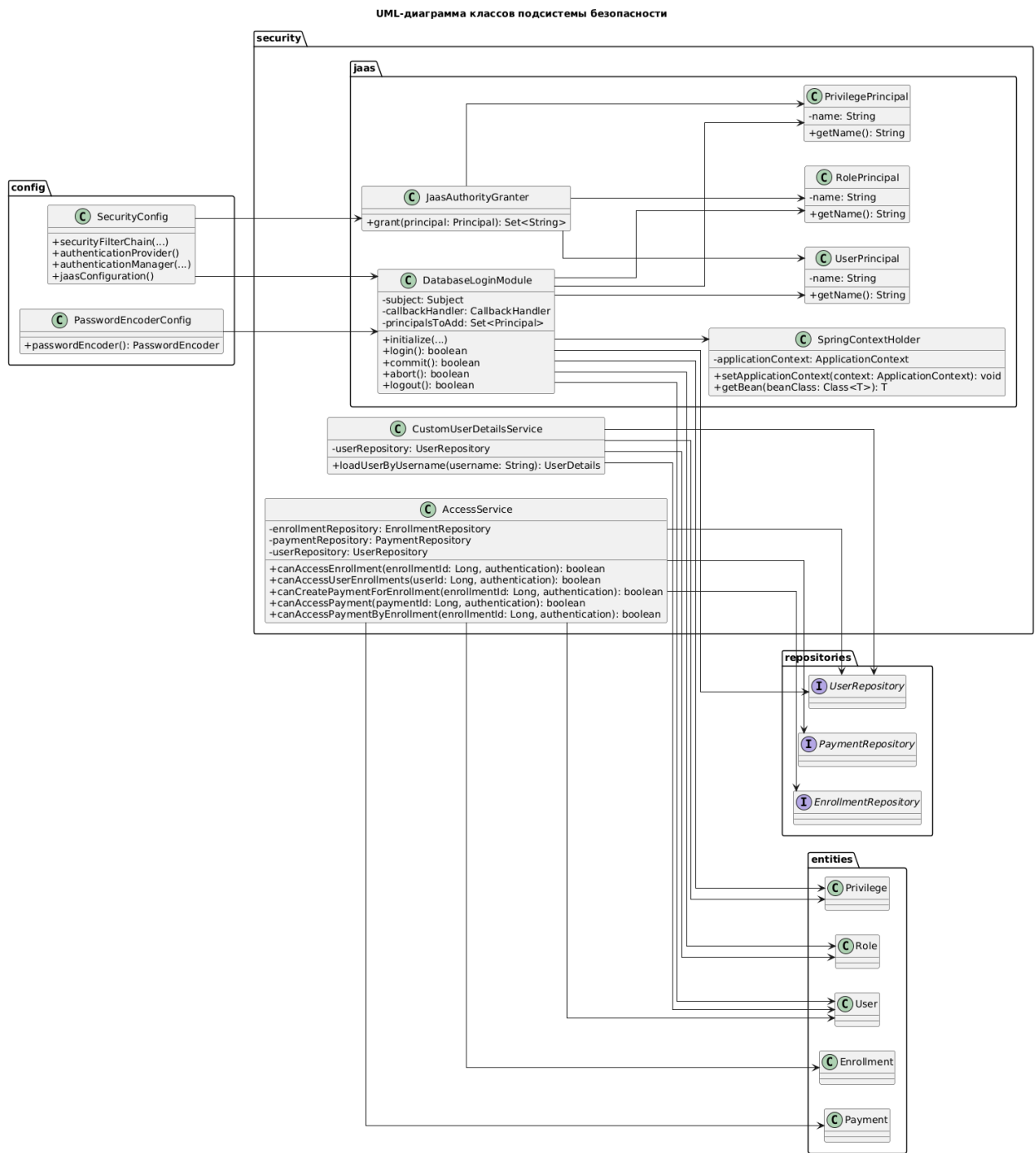


Рис. 5. UML диаграмма классов доступа.

Спецификация REST API для всех публичных интерфейсов разработанного приложения

1. Courses API

Контроллер курсов доступен по пути `/api/courses`. Все операции данного контроллера требуют привилегию `COURSE_READ`. Контроллер предоставляет получение списка курсов с фильтрацией и пагинацией, а также получение курса по идентификатору.

1.1. Получение списка курсов

`GET /api/courses`

Требуемая привилегия: `COURSE_READ`.

Параметр	Тип	Обязательный	Описание
title	String	Нет	Фильтрация курсов по части названия
minPrice	BigDecimal	Нет	Минимальная цена курса, значение не меньше 0.0
maxPrice	BigDecimal	Нет	Максимальная цена курса, значение не меньше 0.0
isActive	Boolean	Нет	Фильтрация по активности курса
onlyAvailable	Boolean	Нет	Если true, возвращаются только курсы со свободными местами
page	Integer	Нет	Номер страницы
size	Integer	Нет	Размер страницы
sort	String	Нет	Поле и направление сортировки

Пример запроса:

```
GET
/api/courses?title=java&minPrice=0&maxPrice=100000&isActive=true&onlyAvailable=true&page=0&size=10
```

Пример успешного ответа:

```
{
  "content": [
    {
      "id": 1,
      "title": "Java Backend Development",
      "description": "Course description",
      "price": 10000.00,
      "currency": "RUB",
      "capacity": 30,
      "availablePlaces": 12,
      "isActive": true
    }
  ],
  "totalElements": 1,
  "totalPages": 1,
  "number": 0,
```

```
"size": 10
}
```

Код	Причина
400 Bad Request	Некорректные query-параметры, например отрицательная цена или minPrice > maxPrice
401 Unauthorized	Пользователь не аутентифицирован
403 Forbidden	У пользователя отсутствует привилегия COURSE_READ

1.2. Получение курса по идентификатору

```
GET /api/courses/{id}
```

Требуемая привилегия: COURSE_READ.

Параметр	Тип	Описание
id	Long	Идентификатор курса. Значение должно быть больше или равно 1

Пример успешного ответа:

```
{
  "id": 1,
  "title": "Java Backend Development",
  "description": "Course description",
  "price": 10000.00,
  "currency": "RUB",
  "capacity": 30,
  "availablePlaces": 12,
  "isActive": true
}
```

Код	Причина
400 Bad Request	Некорректный идентификатор курса
401 Unauthorized	Пользователь не аутентифицирован
403 Forbidden	У пользователя отсутствует привилегия COURSE_READ
404 Not Found	Курс с указанным идентификатором не найден

2. Enrollments API

Контроллер записей на курсы доступен по пути /api/enrollments. Он предоставляет операции создания записи на курс, административного создания записи, а также получения записей по идентификатору, пользователю или курсу. Проверки доступа задаются через привилегии и дополнительную проверку владельца ресурса.

2.1. Создание записи текущего пользователя на курс

```
POST /api/enrollments
```

Требуемая привилегия: ENROLLMENT_CREATE.

Тело запроса:

```
{
  "courseId": 1
}
```

Поле	Тип	Обязательное	Ограничения	Описание
courseId	Long	Да	>= 1	Идентификатор курса

Пример успешного ответа:

```
{
  "id": 10,
  "userId": 1,
  "courseId": 1,
  "status": "PENDING_PAYMENT",
  "rejectionReason": null,
  "paymentExpiresAt": "2026-05-22T12:15:00",
  "activatedAt": null
}
```

Код	Причина
400 Bad Request	Некорректное тело запроса или курс недоступен для записи
401 Unauthorized	Пользователь не аутентифицирован
403 Forbidden	У пользователя отсутствует ENROLLMENT_CREATE
404 Not Found	Пользователь или курс не найден

2.2. Административное создание записи пользователя на курс

POST /api/enrollments/admin

Требуемая привилегия: ENROLLMENT_CREATE_ALL.

Тело запроса:

```
{
  "userId": 1,
  "courseId": 1
}
```

Поле	Тип	Обязательное	Ограничения	Описание
userId	Long	Да	>= 1	Идентификатор пользователя
courseId	Long	Да	>= 1	Идентификатор курса

Пример успешного ответа:

```
{
  "id": 11,
  "userId": 1,
  "courseId": 1,
  "status": "PENDING_PAYMENT",
  "rejectionReason": null,
  "paymentExpiresAt": "2026-05-22T12:15:00",
  "activatedAt": null
}
```

2.3. Получение записи на курс по идентификатору

GET /api/enrollments/{id}

Доступ разрешён, если пользователь имеет привилегию ENROLLMENT_READ_ALL или является владельцем записи.

Параметр	Тип	Описание
id	Long	Идентификатор записи на курс

Пример успешного ответа:

```
{
  "id": 10,
  "userId": 1,
  "courseId": 1,
  "status": "PENDING_PAYMENT",
  "rejectionReason": null,
  "paymentExpiresAt": "2026-05-22T12:15:00",
  "activatedAt": null
}
```

Код	Причина
400 Bad Request	Некорректный идентификатор
401 Unauthorized	Пользователь не аутентифицирован
403 Forbidden	Пользователь не является владельцем записи и не имеет ENROLLMENT_READ_ALL
404 Not Found	Запись не найдена

2.4. Получение записей пользователя

GET /api/enrollments/user/{userId}

Доступ разрешён, если пользователь имеет привилегию ENROLLMENT_READ_ALL или запрашивает собственные записи.

Параметр	Тип	Обязательный	Описание
page	Integer	Нет	Номер страницы
size	Integer	Нет	Размер страницы
sort	String	Нет	Сортировка

Пример запроса:

GET /api/enrollments/user/1?page=0&size=10

2.5. Получение записей по курсу

GET /api/enrollments/course/{courseId}

Требуемая привилегия: ENROLLMENT_READ_ALL.

Параметр	Тип	Обязательный	Описание
page	Integer	Нет	Номер страницы
size	Integer	Нет	Размер страницы
sort	String	Нет	Сортировка

Пример запроса:

GET /api/enrollments/course/1?page=0&size=10

3. Payments API

Контроллер платежей доступен по пути `/api/payments`. Создание платежа требует привилегии `PAYMENT_CREATE` и дополнительной проверки, что запись на курс принадлежит текущему пользователю. Чтение платежа доступно либо пользователю с `PAYMENT_READ_ALL`, либо владельцу платежа.

3.1. Создание платежа по записи на курс

```
POST /api/payments/enrollment/{enrollmentId}
```

Требуемые права: `PAYMENT_CREATE` и принадлежность `enrollment` текущему пользователю.

Параметр	Тип	Описание
<code>enrollmentId</code>	Long	Идентификатор записи на курс

Тело запроса не требуется.

Пример успешного ответа:

```
{
  "id": 20,
  "enrollmentId": 10,
  "providerPaymentId": "provider-payment-123",
  "amount": 10000.00,
  "currency": "RUB",
  "status": "CREATED",
  "paymentUrl": "http://localhost:1489/payments/provider-payment-123",
  "failureReason": null,
  "expiresAt": "2026-05-22T12:01:00",
  "paidAt": null
}
```

Код	Причина
400 Bad Request	Платёж уже существует, <code>enrollment</code> не в статусе <code>PENDING_PAYMENT</code> , ошибка внешнего банковского сервиса
401 Unauthorized	Пользователь не аутентифицирован
403 Forbidden	У пользователя нет <code>PAYMENT_CREATE</code> или <code>enrollment</code> ему не принадлежит
404 Not Found	<code>Enrollment</code> не найден

3.2. Получение платежа по идентификатору

```
GET /api/payments/{id}
```

Доступ разрешён, если пользователь имеет привилегию `PAYMENT_READ_ALL` или является владельцем платежа.

Параметр	Тип	Описание
<code>id</code>	Long	Идентификатор платежа

Пример успешного ответа аналогичен ответу операции создания платежа.

3.3. Получение платежа по идентификатору записи на курс

```
GET /api/payments/enrollment/{enrollmentId}
```

Доступ разрешён, если пользователь имеет привилегию PAYMENT_READ_ALL или является владельцем записи на курс.

Параметр	Тип	Описание
enrollmentId	Long	Идентификатор записи на курс

Пример успешного ответа аналогичен ответу операции создания платежа.

4. Payment Webhook API

Контроллер webhook-запросов доступен по пути /api/payments/webhook. Данный публичный интерфейс предназначен для получения callback-запросов от внешнего банковского сервиса. Endpoint принимает статус платежа от провайдера и запускает обработку результата оплаты в рамках программно управляемой транзакции.

Поддерживаются статусы PAID и FAILED.

4.1. Обработка callback-запроса от банковского сервиса

POST /api/payments/webhook

Требуемая привилегия: PAYMENT_CALLBACK_HANDLE.

Тело запроса:

```
{
  "providerPaymentId": "provider-payment-123",
  "status": "PAID",
  "failureReason": null
}
```

Поле	Тип	Обязательное	Ограничения	Описание
providerPaymentId	String	Да	Не пустое, максимум 100 символов	Идентификатор платежа во внешнем банковском сервисе
status	String	Да	Не пустое, максимум 32 символа	Статус платежа. Поддерживаются PAID и FAILED
failureReason	String	Нет	Максимум 255 символов	Причина неуспешной оплаты

Пример успешной оплаты:

```
{
  "providerPaymentId": "provider-payment-123",
  "status": "PAID"
}
```

Пример неуспешной оплаты:

```
{
  "providerPaymentId": "provider-payment-123",
  "status": "FAILED",
  "failureReason": "Card declined in test scenario"
}
```

Успешный ответ: 200 OK, тело ответа отсутствует.

Код	Причина
-----	---------

400 Bad Request	Некорректное тело запроса, пустой providerPaymentId, пустой status или неподдерживаемый статус платежа
401 Unauthorized	Пользователь или сервис не аутентифицирован
403 Forbidden	Отсутствует привилегия PAYMENT_CALLBACK_HANDLE
404 Not Found	Платёж с указанным providerPaymentId не найден

5. Сводная таблица публичных endpoint-ов

Метод	URL	Описание	Требуемые права
GET	/api/courses	Получение списка курсов	COURSE_READ
GET	/api/courses/{id}	Получение курса по ID	COURSE_READ
POST	/api/enrollments	Создание собственной записи на курс	ENROLLMENT_CREATE
POST	/api/enrollments/admin	Создание записи на курс для пользователя	ENROLLMENT_CREATE_ALL
GET	/api/enrollments/{id}	Получение записи на курс по ID	ENROLLMENT_READ_ALL или владелец записи
GET	/api/enrollments/user/{userId}	Получение записей пользователя	ENROLLMENT_READ_ALL или сам пользователь
GET	/api/enrollments/course/{courseId}	Получение записей по курсу	ENROLLMENT_READ_ALL
POST	/api/payments/enrollment/{enrollmentId}	Создание платежа	PAYMENT_CREATE и владелец enrollment
GET	/api/payments/{id}	Получение платежа по ID	PAYMENT_READ_ALL или владелец платежа
GET	/api/payments/enrollment/{enrollmentId}	Получение платежа по enrollment	PAYMENT_READ_ALL или владелец enrollment
POST	/api/payments/webhook	Обработка callback от банка	PAYMENT_CALLBACK_HANDLE

Исходный код системы или ссылка на репозиторий с исходным кодом

[Ссылка на репозиторий с кодом основного приложения.](#)

[Ссылка на репозиторий с кодом банковского приложения.](#)

Вывод по работе

В ходе выполнения лабораторной работы было доработано приложение, разработанное в рамках лабораторной работы №1. Основными направлениями доработки стали: внедрение механизма управления распределёнными транзакциями с использованием Spring JTA и Atomikos, а также реализация разграничения доступа к операциям бизнес-логики на базе Spring Security и JAAS с хранением учётных записей в реляционной базе данных. Также были внесены изменения, исходя из замечаний на защите лабораторной работы №1: исправлены некоторые коды ответа при валидации; добавлена пагинация, фильтрация и батчинг для основных сущностей.